1. Lsqlin

1.1. Cél

Nem túlterhelő állapotot létrehozó csomóponti betáplálásokat és vételezéseket tartalmazó vektor előállítása, felhasználva az eredetileg általunk definiált vektort.

Tehát úgy módosítjuk, hogy a referencia állapotban:

- ne legyen szükség virtuális kapacitásra, vagyis AMR-re,
- valamint az lsqlin metodikája miatt módosított értékek minél kevésbé térjenek el az eredetileg megadottaktól.

A számítást a MATLAB `lsqlin` (legkisebb négyzetes eltérést optimalizáló) függvénye végzi. Az ehhez tartozó szekció alapesetben ki van kommentezve.

1.2. Az optimalizálási feladat

A hálózat 15 csomópontjából a slack csomópont nem szerepel önálló változóként (mivel az a többi kiegyenlítéséért felel), így 14 csomópont értékét keresi a függvény.

Legyen:

- x_{ref} az eredeti, slack nélküli csomóponti vektor;
- x a keresett új vektor;
- $PTDF_{node,CBCO}$ a vizsgált CBCO-k csomóponti PTDF-mátrixa.

A vizsgált CBCO itt azt jelöli, hogy ebben az esetben csak az alapesetet és 6 kiesést ["L1", "L2","L7","L20", "L21","L22"] tartalmazó a CBCO tábla lett figyelembe véve. Viszont az lsqlin kimenete így is egy nem túlterhelő referencia pontot biztosított, szóval ez a 6 kiesés elégnek bizonyult. Természetesen a többi esetnél (pl.: validáció) már a teljes CBCO tábla lett előállítva és figyelembe véve.

A számított áramlások:

$$F = PTDF_{node,CBCO} \cdot x. \tag{1}$$

A megengedett áramlás:

$$Limit = F_{max} - FRM. \tag{2}$$

Ezek után a definiált feladat:

$$\min_x \quad \|x - x_{ref}\|_2^2 \tag{3}$$

$$\text{ha} \quad PTDF_{node,CBCO} \cdot x \leq Limit, \tag{4}$$

$$-PTDF_{node,CBCO} \cdot x \leq Limit, \tag{5}$$

$$-1000 \leq x_i \leq 1000. \tag{6}$$

Az utolsónál az alsó és felső korlát önkényesen lett megválasztva.

A célfüggvény az egyes csomóponti módosítások négyzetösszege:

$$\|x - x_{ref}\|_2^2 = \sum_{i=1}^{14} (x_i - x_{ref,i})^2. \tag{7}$$

A két áramlási feltétel a pozitív és negatív irányt is korlátozza:

$$|F_c| \leq F_{max,c} - FRM_c. \tag{8}$$

Külön F_0 tag nincs, mert a csomóponti PTDF és a teljes csomóponti vektor közvetlenül az áramlást adja meg.

1.3. Kódbeli megvalósítás

Az `lsqlin` célfüggvénye $\|Cx - d\|_2^2$ alakú, amelyben C egy egységmátrixszal helyettesíthető:

$$C = I, \qquad d = x_{ref}. \tag{9}$$

A korlátok összeállítása:

```
x_ref = np;

Aineq = [
     ntsPTDF_cbco;
    -ntsPTDF_cbco
];

[~,edgeIndex] = ismember( ...
    string(CBCO_table.CriticalBranch), ...
    string(edges.EdgeName));

limit = edges.Fmax(edgeIndex) ...
      - edges.FRM(edgeIndex);
```

```
bineq = [limit; limit];

C = eye(numel(x_ref));
d = x_ref;

lb = -1000*ones(numel(x_ref),1);
ub =  1000*ones(numel(x_ref),1);

[x_new,resnorm,residual,exitflag,output] = ...
    lsqlin(C,d,Aineq,bineq,[],[], ...
           lb,ub,[],options);
```

Az `edgeIndex` azt biztosítja, hogy minden CBCO-sorhoz a megfelelő monitorozott vezeték határértékét használjuk.

Fontos, hogy jobb oldalon az $F_{max} - FRM$ értékek szerepelnek, nem az AMR-rel kiegészített piaci RAM-ok.

1.4. A slack visszaszámítása

A megoldás után a slack csomópont értéke:

$$np_{slack} = -\sum_{i \neq slack} np_i. \tag{10}$$

Így a teljes hálózat egyensúlyban lesz:

$$\sum_{i=1}^{15} np_i = 0. \tag{11}$$

A kódban:

```
nodeNP_new = zeros(height(nodes),1);

nodeNP_new(nodes.NodeName ~= slacknodename) ...
    = x_new;

nodeNP_new(slacknodeid) = -sum(x_new);
```

A 14 változóra ezért nem is kellett külön egyensúly-feltétel, mivel a különbséget a slack egyenlíti ki.

Az utóellenőrzésnél is a `limit` vektorhoz kell hasonlítani az áramlásokat. A kommentezett ellenőrző táblázatban szereplő `finalram` vektorok erre nem megfelelőek.

2. Presolve

2.1. Cél

A piaci domain redundáns korlátainak elhagyása, tehát azon constrainteket, melyek teljesülését szigorúbb korlátok biztosítják. Azaz a domain alakja nem változik, csak kevesebb egyenlőtlenséggel lesz leírva.

2.2. Kiinduló domain

A négy zóna nettópozíció-vektora:

$$NP = \begin{bmatrix} NP_A & NP_B & NP_C & NP_D \end{bmatrix}^{\mathsf{T}}. \tag{12}$$

Az utolsó feltétel vonatkozik a nettó poziciók egyensúlyára (összegük 0). A piaci feltételek:

$$PTDF_{CBCO} \cdot NP \leq RAM^{+}, \tag{13}$$
$$-PTDF_{CBCO} \cdot NP \leq RAM^{-}, \tag{14}$$
$$1^{\mathsf{T}} NP = 0. \tag{15}$$

A két irányt egymás alá írjuk:

$$A_{domain} = \begin{bmatrix} PTDF_{CBCO} \\ -PTDF_{CBCO} \end{bmatrix}, \qquad b_{domain} = \begin{bmatrix} RAM^{+} \\ RAM^{-} \end{bmatrix}. \tag{16}$$

A teljes CBCO-listában:

$$N_{CBCO} = 22 + 22 \cdot 21 = 484. \tag{17}$$

A két irány miatt összesen 968 korlát van.

2.3. Egy korlát vizsgálata

Az i-edik korlát:

$$A_{domain}(i,:) \cdot NP \leq b_{domain,i}. \tag{18}$$

Azt vizsgáljuk, hogy csak a többi korlátot figyelembe véve meg lehet-e sérteni ezt a feltételt. Akkor redundáns, ha a válasz nem.

Ehhez a vizsgált korlát jobb oldalát ideiglenesen 1 MW-tal lazítjuk, ezzel lehetőséget biztosítva az esetleges túlterhelésre:

$$b_{test,i} = b_{domain,i} + 1. \tag{19}$$

Ezután a bal oldalt maximalizáljuk:

$$M_i = \max_{NP} A_{domain}(i,:) \cdot NP. \tag{20}$$

A maximumkeresésben az aktuálisan megtartott korlátok, valamint a zónás nettó pozíció egyensúly szerepel, a vizsgált sornál viszont a lazított jobb oldal van figyelembe véve.

Ha:

$$M_i \leq b_{domain,i}, \tag{21}$$

azaz a maximum az eredeti határ alatt marad,, akkor a többi CBCO a lazítás ellenére sem engedte meg a túlterhelést, ezért a vizsgált korlát elhagyható. Ellenkező esetben a vizsgált sor nem redundáns. Ezen gondolatmenet hitelességét a domain konvexitása biztosítja.

2.4. Kódbeli megvalósítás

Kezdetben tehát minden korlátot megtartunk:

```
keep = true(nConstraint,1);
```

A vizsgálat során pedig végigmegyünk az összes CBCO soron:

```
for i = 1:nConstraint

    rows = find(keep);

    A_test = A_domain(rows,:);
    b_test = b_domain(rows);

    selected = find(rows == i);

    b_test(selected) = b_test(selected)+1;

    [~,fval] = linprog( ...
        -A_domain(i,:).', ...
        A_test,b_test, ...
        ones(1,zonenum),0, ...
        [],[],options);

    maximum(i) = -fval;

    if maximum(i) <= b_domain(i)+tol
```

```
        keep(i) = false;
    end
end
```

Mivel a `linprog` minimalizál, ezért a maximumkereséshez negatív célfüggvény került megadásra. A keresett maximum így `-fval`.

Az is meg lett oldva, hogy a törlés dinamikusan történjen, vagyis a korábban redundánsnak megállapított sorokat a következő vizsgálatok már nem használják.

A megmaradó rendszer:

```
A_presolve = A_domain(keep,:);
b_presolve = b_domain(keep);
```

Helyes redundanciadöntések esetén:

$$\mathcal{D}_{presolve} = \mathcal{D}_{original}. \tag{22}$$

A `presolve_table` minden eredeti sorhoz eltárolja a számított maximumot és azt, hogy a sor megmaradt-e.

2.5. Fontosabb változók

Változó	**Jelentés**
`A_domain`, `b_domain`	Az eredeti piaci korlátrendszer.
`keep`	A megtartott sorokat jelölő logikai vektor.
`rows`	Az aktuálisan megtartott sorok indexei.
`selected`	A vizsgált sor helye az aktuális listában.
`A_test`, `b_test`	Az ideiglenes vizsgálati feladat adatai.
`maximum`	A sorok vizsgálatakor kapott maximumok.
`A_presolve`, `b_presolve`	Az egyszerűsített korlátrendszer.
`presolve_table`	A vizsgálati eredmények táblázata.
`presolve_result`	Az eredményeket összefogó struktúra.

3. Teljes domain validálása

3.1. Cél

A virtuális kapacitásokkal kiegészített piaci domainben szereplő túlterhelő pontok megtalálása, majd korrigálása IVA-val, szűkítve ezzel a domaint.

A teljes validáció algoritmusa:

1. Minden CBCO mindkét irányában megkeressük a domainen belüli legnagyobb áramlást.
2. Ezután kiválasztjuk a legnagyobb határsértést.
3. Ezt követően szigorítjuk az ehhez tartozó piaci korlátot, IVA-val, aminek nagysága megegyezik az eredetileg kiszabott AMR-rel.
4. Majd újraszámítjuk a maximumokat az új domainben (mivel egy CBCO változtatása más túlterhelő pontot is korrigálhat).
5. E folyamat ismétlése, amíg nem marad határsértés.

A megengedett áramlás:

$$Limit_c = F_{max,c} - FRM_c. \tag{23}$$

3.2. Teljes CBCO összeállítása

A függvény az alapesetet és az összes egyszeres vezetékkiesést vizsgálja.

A CBCO-lista hossza:

$$N_{CBCO} = 22 + 22 \cdot 21 = 484. \tag{24}$$

A pozitív és negatív irány együtt 968 korlátot ad.

Egy o vezeték kiesésekor:

$$PTDF_{c,o} = PTDF_c + LODF_{c,o} \cdot PTDF_o, \tag{25}$$

$$F_{ref,c,o} = F_{ref,c} + LODF_{c,o} \cdot F_{ref,o}. \tag{26}$$

Módosított F0:

$$F_{0,c,o} = F_{ref,c,o} - PTDF_{c,o} NP_{ref}. \tag{27}$$

Egy CBCO áramlása ezért:

$$F_c(NP) = F_{0,c} + PTDF_c NP. \tag{28}$$

A képletekben persze az adott kiesésre vonatkozó LODF-ek szerepelnek.

3.3. Az eredeti piaci domain

Az AMR nélküli RAM:

$$RAM^{+}_{0,c} = Limit_c - F_{0,c}, \qquad RAM^{-}_{0,c} = Limit_c + F_{0,c}. \tag{29}$$

A minimum-RAM és az AMR:

$$minRAM_c = Ramr \cdot F_{max,c}, \tag{30}$$
$$AMR^{\pm}_c = \max(minRAM_c - RAM^{\pm}_{0,c}, 0), \tag{31}$$
$$RAM^{\pm}_c = RAM^{\pm}_{0,c} + AMR^{\pm}_c. \tag{32}$$

A `Ramr` a minimum-RAM követelmény aránya a vezeték terhelhetőségéhez képest; értéke 0,70.

A piaci feltételek közös alakja:

$$A_{domain} = \begin{bmatrix} PTDF_{CBCO} \\ -PTDF_{CBCO} \end{bmatrix}, \qquad b_{domain} = \begin{bmatrix} RAM^{+} \\ RAM^{-} \end{bmatrix}. \tag{33}$$

A domain:

$$\mathcal{D} = \left\{ NP \mid A_{domain} \cdot NP \leq b_{domain}, \quad 1^{\mathsf{T}} NP = 0 \right\}. \tag{34}$$

3.4. A legnagyobb áramlások keresése

Minden CBCO-ra két LP-t kell megoldani, egyet a pozitívra, egyet a negatívra. A két LP után tudjuk megállapítani a legrosszabb esetet, amelyet kezelni kell az IVA-val.

Pozitív irány:

$$M^{+}_c = F_{0,c} + \max_{NP \in \mathcal{D}} PTDF_c \cdot NP. \tag{35}$$

Negatív irány:

$$M^{-}_c = -F_{0,c} + \max_{NP \in \mathcal{D}} (-PTDF_c \cdot NP). \tag{36}$$

Az M^{-}_c a negatív irány terhelését, vagyis a $-F_c$ maximumát jelenti.

A pozitív irány kódban:

```
[NPplus,fval_plus] = linprog( ...
    -PTDF_cbco(c,:).', ...
    A_domain,b_modified, ...
    ones(1,4),0, ...
    -inf(4,1),inf(4,1),options);
```

A negatív irányban a célfüggvény:

```
PTDF_cbco(c,:).'
```

Mivel a solver minimalizál:

$$M_c^+ = F_{0,c} - fval_c^+, \qquad M_c^- = -F_{0,c} - fval_c^-. \tag{37}$$

Az F_0 az adott CBCO-nál állandó, ezért csak az optimalizálás után adjuk hozzá. A maximumhelyet nem befolyásolja.

3.5. Domain szűkítése

A két irány maximumaiból kiválasztjuk a legnagyobb határsértést okozót. A kódban az alábbi logikával lett ez a lépés megvalósítva:

$$current_max = \begin{bmatrix} M^+ \\ M^- \end{bmatrix}. \tag{38}$$

$$excess = current_max - \begin{bmatrix} Limit \\ Limit \end{bmatrix}. \tag{39}$$

```
[largest_excess,selected] = max(excess);
```

Ha `largest_excess <= tol`, azaz nincs toleranciánál nagyobb határsértés, az algoritmus leáll.

Egyébként a kiválasztott sor jobb oldalát a fizikai RAM-ra csökkentjük:

$$b_{physical} = \begin{bmatrix} Limit - F_0 \\ Limit + F_0 \end{bmatrix}. \tag{40}$$

A kódban:

```
old_RAM = b_modified(selected);

new_RAM = min( ...
    old_RAM,b_physical(selected));

b_modified(selected) = new_RAM;

IVA(selected) = ...
    b_domain(selected)-b_modified(selected);
```

Lényegében a kiválasztott irány RAM-ját a RAM0-ra állítjuk, vagyis ezen a soron visszavonjuk az AMR-t.

3.6. IVA

Az IVA az eredeti piaci RAM csökkentése:

$$IVA_i = b_{domain,i} - b_{modified,i}. \tag{41}$$

Így:

$$RAM_{validated,i} = RAM_{0,i} + AMR_{market,i} - IVA_i. \tag{42}$$

Nem kell minden soron közvetlenül visszavonni az AMR-t. Egy másik korlát szigorítása már kizárhatja az adott sor veszélyes pontjait. Erre volt az újraszámolás lépése az algoritmusban.

Ezentúl a kód még külön ellenőrzi, hogy nem üres-e a módosított domain, illetve minden végső irányított maximum a határ alatt marad-e.

Ezek eredménye a `SafeDomainFeasible` és `SafeDomainVerified` mező.

3.7. Fontos változók

Változó	**Jelentés**
`PTDF_cbco`	A teljes CBCO-lista zónás PTDF-mátrixa.
`Fref`, `F0`	Referenciaáramlás és állandó áramlási tag.
`flow_limit`	Az $F_{max} - FRM$ áramlási határ.
`A_domain`, `b_domain`	Az eredeti piaci domain adatai.
`b_physical`	A fizikai feltételekből számított RAM0-vektor.
`b_modified`	Az aktuális RAM-vektor.
`current_max`	Az aktuális domain irányonkénti maximumai.
`worstNP_plus/minus`	A maximumokat előidéző piaci pontok.
`excess`	Az irányonkénti határsértés.
`selected`	A következő szigorítandó irány indexe.
`IVA`	Az eredeti RAM-hoz képesti csökkentés.
`result.IVATable`	RAM- és IVA-adatok minden irányra.
`result.CorrectedTable`	A közvetlenül módosított sorok.
`result.RestrictionHistory`	A módosítások sorrendje és mértéke.
`result.SafeResultsTable`	A kezdeti és végső maximumok összehasonlítása.

4. Lokális validáció a referenciaprogram környezetében

4.1. Cél

A lokális validáció ugyanazt a maximumkeresési és piacszűkítési logikát használja, mint a teljes validáció, viszont a referenciaprogram egy adott környezetében vizsgálódik.

Ez a kód is az eredeti `result.b_domain` vektorból indul.

4.2. A környezet megadása

A környezetet egy refprog körüli gömbnek vesszük, amelynél a gömb sugarát definiáljuk:

$$\|NP - NP_{ref}\|_2 \leq R. \tag{43}$$

Kifejtve:

$$\sum_{z=1}^{4} (NP_z - NP_{ref,z})^2 \leq R^2. \tag{44}$$

Ez tehát az összes zóna együttes eltérését korlátozza.

A kódban normalizált alak szerepel:

```
local_constraint = @(NP) deal( ...
    sum((NP-NP_ref).^2)/radius^2-1, []);
```

A tényleges vizsgálati tartomány tehát a gömb és az eredeti domain metszete:

$$\mathcal{D}_{local}^{(k)} = \mathcal{D}_{market}^{(k)} \cap \{NP \mid \|NP - NP_{ref}\|_2 \leq R\}. \tag{45}$$

A k az iterációt jelöli: a piaci domain a RAM-csökkentések miatt változik.

Ebből adódóan egy fontos különbség a teljes validáláshoz képest, hogy itt kvadratikus korlátok is megjelennek, emiatt ez a problémakör már nem LP-nek tekinthető. Ennek következtében más solvereket kell alkalmazni

4.3. Kezdőpont keresése

Minden iteráció elején megkeressük az aktuális lineáris piaci domain referenciához legközelebbi pontját:

$$\min_{NP} \quad \frac{1}{2}\|NP - NP_{ref}\|_2^2 \tag{46}$$

$$\text{mellett} \quad A_{domain} \cdot NP \leq b_{modified}, \tag{47}$$

$$1^{\mathsf{T}} \cdot NP = 0. \tag{48}$$

A kezdőpontkeresés célfüggvénye kvadratikus, a piaci korlátok és a zónás egyensúly viszont lineárisak. Ezért ez egy konvex kvadratikus programozási feladat, amelyet a `quadprog` függvénnyel oldunk meg.

A kezdőpontot azért számítjuk ki minden iterációban, mert a RAM-ok csökkentése után a korábbi kezdőpont már nem feltétlenül teljesíti az aktuális piaci feltételeket. A `quadprog` ilyenkor a referenciához legközelebbi, az új domainben megengedett pontot keresi meg.

Ez csak a későbbi maximumkeresés kezdőpontja: a referenciaprogram és a vizsgált környezet középpontja nem változik.

```
[NP_start,~] = quadprog( ...
    eye(zonenum),-NP_ref, ...
    A_domain,b_modified, ...
    ones(1,zonenum),0, ...
    [],[],[],options_start);
```

A célfüggvényből a konstans tagot elhagyva:

$$\frac{1}{2}NP^{\mathsf{T}} \cdot NP - NP_{ref}^{\mathsf{T}} \cdot NP. \tag{49}$$

Ezért szerepel egységmátrix és `-NP_ref` a függvényhívásban.

Ha a domain és a környezet közös része nem üres, akkor ez a legközelebbi pont a környezeten belül lesz. Ha a legkisebb távolság is nagyobb R-nél, a lokális tartomány üres.

4.4. Lokális maximumkeresés

A pozitív irányú maximum:

$$M_c^+ = F_{0,c} + \max_{NP \in \mathcal{D}_{local}} PTDF_c \cdot NP. \tag{50}$$

A negatív irányú maximum:

$$M_c^- = -F_{0,c} + \max_{NP \in \mathcal{D}_{local}} (-PTDF_c \cdot NP). \tag{51}$$

A maximumkeresésnél a célfüggvény lineáris, de a referenciától való távolságot kvadratikus egyenlőtlenség korlátozza. A feladat ezért nem oldható meg közvetlenül a teljes validációnál használt `linprog` függvénnyel, amely lineáris feltételeket kezel.

A `quadprog` sem alkalmazható közvetlenül erre a felírásra: az kvadratikus célfüggvényt kezel lineáris korlátok mellett, itt viszont a kvadratikus kifejezés a korlátban szerepel.

Ezért a maximumkereséshez a `fmincon` függvényt használjuk, amely nemlineáris korlátokat is fogad. A piaci egyenlőtlenségeket és a zónás egyensúlyt lineáris feltételként, a gömbfeltételt pedig a `local_constraint` függvényen keresztül adjuk át.

A `fmincon` a `quadprog` által előállított `NP_start` pontból indul, és azon belül keresi az adott CBCO legnagyobb áramlását, ahol a piaci és a távolsági feltételek egyszerre teljesülnek. A keresés az `interior-point` algoritmus szerint működik.

Bár általános nemlineáris megoldót alkalmazunk, a jelen feladat mégis konvex, ezért egy megtalált szélső érték automatikusan globális optimum is lesz.

Pozitív irány:

```
[NPplus,fval_plus] = fmincon( ...
    @(NP) -PTDF_cbco(c,:)*NP, ...
    NP_start, ...
    A_domain,b_modified, ...
    ones(1,zonenum),0, ...
    [],[],local_constraint,options_local);
```

Negatív irány:

```
[NPminus,fval_minus] = fmincon( ...
    @(NP) PTDF_cbco(c,:)*NP, ...
    NP_start, ...
    A_domain,b_modified, ...
    ones(1,zonenum),0, ...
    [],[],local_constraint,options_local);
```

Az eredményekből:

$$M_c^+ = F_{0,c} - fval_c^+, \qquad M_c^- = -F_{0,c} - fval_c^-. \tag{52}$$

4.5. Algoritmus lépései

A `while true` ciklus minden iterációjában a következő lépések történnek:

1. A `quadprog` meghatározza az aktuális piaci domain referenciához legközelebbi pontját, amely a maximumkeresések kezdőpontja lesz.
2. A `fmincon` minden CBCO mindkét irányában megkeresi a legnagyobb áramlást a domain és a referenciakörnyezet közös részében.
3. Eltároljuk a maximumokat és az azokat előidéző zónás nettópozíció-vektorokat.
4. Meghatározzuk a legnagyobb határsértést és a hozzá tartozó irányított korlátot.
5. Ha a legnagyobb sértés sem haladja meg a toleranciát, a ciklus leáll. Ellenkező esetben a kiválasztott sor RAM-ját a hozzá tartozó RAM0-ra csökkentjük, és mentjük a beavatkozás adatait.
6. A módosított domainnel új iteráció indul, amelyben a kezdőpontot és az összes lokális maximumot újraszámítjuk.

Az irányonkénti sértések:

$$excess = \begin{bmatrix} M^+ \\ M^- \end{bmatrix} - \begin{bmatrix} L \\ L \end{bmatrix}, \qquad L = F_{max} - FRM. \tag{53}$$

A legnagyobb sértéshez tartozó sort választjuk:

```
[largest_excess,selected] = max(excess);
```

Ha nincs toleranciánál nagyobb sértés, az algoritmus leáll. Egyébként:

```
old_RAM = b_modified(selected);

new_RAM = min( ...
    old_RAM,b_physical(selected));

b_modified(selected) = new_RAM;
```

A következő körben újraszámítjuk a kezdőpontot és minden CBCO mindkét lokális maximumát.

A referencia és a környezet középpontja közben nem változik. Az új kezdőpont nem jelent új referenciaprogramot.

Az IVA:

$$IVA_i = b_{domain,i} - b_{modified,i}. \tag{54}$$

A `history` a korrekciók adatait, a `history_NP` pedig a korrekciókat kiváltó piaci pontokat tárolja.

Sikeres maximumkeresések és befejezés esetén:

$$|F_c(NP)| \leq F_{max,c} - FRM_c \tag{55}$$

a módosított domain és a környezet közös részében.

A környezeten kívül maradhatnak határsértő pontok.

4.6. Fontos változók

Változó	**Jelentés**
`NP_ref`	A rögzített referenciaprogram.
`radius`	A környezet sugara: 250 MW.
`local_constraint`	A kvadratikus távolsági korlát.
`NP_start`	Az aktuális domainhez számított kezdőpont.
`b_modified`	Az aktuális RAM-vektor.
`max_plus`, `max_minus`	A lokális irányított maximumok.
`worstNP`	Az aktuális maximumhelyek.
`original_max`	A kezdeti lokális maximumok.
`current_max`	Az aktuális lokális maximumok.
`excess`, `selected`	A sértések és a legnagyobb sértés indexe.
`history`	A korrekciók sorrendje és RAM-adatai.
`history_NP`	A korrekciókat kiváltó piaci pontok.
`local_result.IVA`	Az eredeti RAM-hoz képesti csökkentés.
`local_result.ResultsTable`	A kezdeti és végső lokális eredmények.